Multimodal Crop Health Monitoring: A Modular ResNet-18 + Rule-Based Risk Layer

Khang Phan
EGN6217 – Applied Deep Learning
Spring 2026
github.com/itsmekhang/crop-health-monitoring

***Abstract*—This paper documents the first full implementation of a modular crop health system that combines an image-based disease classifier with a weather-informed risk layer. A ResNet-18 model fine-tuned on PlantVillage classifies leaf disease across 15 classes covering Tomato, Potato, and Pepper. An XGBoost model then approximates a rule-based risk formula using the disease output and live weather conditions from the Open-Meteo API. Both models are integrated into a Streamlit dashboard that provides real-time diagnosis, environmental risk assessment, and estimated revenue loss. This report documents the implementation decisions, an honest evaluation of both models, and the key challenges identified during this phase, particularly class imbalance, the lab-to-field generalization gap, and the derived nature of the XGBoost training target.**

I. Project Summary

A. Purpose

Farmers still rely on manual disease scouting or delayed laboratory testing, leading to late intervention and unnecessary crop losses. This project builds a modular AI prototype that processes a leaf photograph and a location, and returns a disease diagnosis, an environmental risk score, a treatment plan, and an estimated revenue impact. The goal is a functioning, honest prototype – not an overclaimed production system.

B. Progress Since Deliverable 1

Deliverable 1 established the project scope, planned architecture, and data sources. Since then, the following has been completed:

- ResNet-18 disease classifier trained and saved to `results/disease_model.pth`
- XGBoost environmental risk model trained and saved to `results/risk_model.pkl`
- Streamlit interface (`ui/app.py`) integrating both models with live weather
- Per-class accuracy evaluation and confusion matrix added to the training notebook
- Dataset scope corrected from 38 to 15 classes; class imbalance ($21\times$) documented
- XGBoost framing revised: now explicitly described as a rule approximator with a derived target

C. What Remains

The following improvements are planned before Deliverable 3: running the full per-class accuracy table against the trained model and reporting those numbers; testing the interface with real field photos to quantify the lab-to-field gap; and exploring whether fine-tuning deeper ResNet layers improves accuracy on minority classes.

II. System Architecture and Pipeline

Fig. 1 shows the full data flow. The two models operate independently and are combined only at the output stage. This is a deliberate design choice: it avoids the alignment problem of true multimodal fusion and keeps each component auditable.

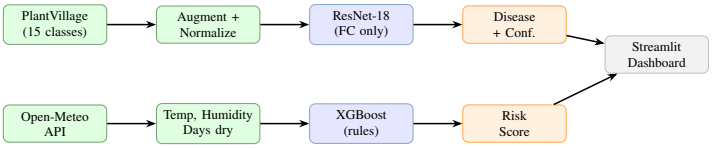

Fig. 1. System pipeline. ResNet-18 processes the leaf image independently of weather; XGBoost combines the disease output with environmental features.

A. Model Roles

ResNet-18 is the academic center of the project. It performs the primary classification task: given a leaf image, predict which of the 15 disease or healthy classes it belongs to. Weather does not enter this model – a diseased leaf looks the same regardless of humidity.

XGBoost is a decision-support layer. It takes the predicted disease class, the classifier confidence, and three weather features (temperature, humidity, days since rain) and outputs a risk score in $[0, 1]$. This score drives the urgency level and treatment recommendation displayed in the UI. The training target is derived from agronomic rules in `src/fusion.py`, not from field observations (see Section V).

III. Model Implementation Details

A. Disease Classifier – ResNet-18

Framework: PyTorch 2.x with torchvision.

Architecture: ResNet-18 initialized with ImageNet weights. All convolutional layers are frozen; only the final fully connected layer is fine-tuned. The replacement head maps from 512 features to $C = 15$ classes.

Training setup:

- Hardware: CPU (or CUDA if available)
- Epochs: 10

- Batch size: 32
- Optimizer: Adam, lr $= 10^{-3}$
- Scheduler: StepLR, step_size $= 3$, $\gamma = 0.5$
- Loss: CrossEntropyLoss

Dataset split: Two separate `ImageFolder` instances are constructed from `data/raw/PlantVillage` – one with training augmentations, one with only normalization. A `Subset` object with a fixed random permutation (seed 42) assigns 80% to training and 20% to validation. Using two instances avoids the transform contamination that occurs when a single `ImageFolder` object has its `.transform` reassigned mid-training.

Augmentations: Random horizontal flip, rotation $\pm 15°$, color jitter (brightness/contrast ± 0.2), then ImageNet normalization.

Reproducibility: The training notebook (`notebooks/setup.ipynb`) runs end-to-end from dataset download to model save. All random seeds are fixed.

B. Environmental Risk Layer – XGBoost

Framework: XGBoost 2.x with scikit-learn API.

Architecture: XGBRegressor with 300 estimators, max_depth $= 5$, learning rate $= 0.05$, subsample $= 0.8$, colsample_bytree $= 0.8$.

Features: temperature (°C), humidity (%), days since rain, classifier confidence, label-encoded disease class.

Target construction: The training set is generated synthetically. For each disease class, 500 (disease, weather) samples are drawn from disease-specific weather distributions (e.g., fungal diseases are sampled with high humidity). The risk score label is then computed by the deterministic formula in `src/fusion.py`:

$$s = 0.4 \cdot b_{\text{class}} + 0.3 \cdot c + 0.3 \cdot f_{\text{env}}(t, h, r) \tag{1}$$

where b_{class} is a fixed per-class severity base, c is classifier confidence, and f_{env} is a weighted environmental score.

This derivation is the key caveat for interpreting XGBoost performance (Section V).

C. Interface

The Streamlit dashboard (`ui/app.py`) accepts one or more leaf image files, a location string (geocoded via the Open-Meteo API), and economic inputs (field size in acres, crop price per lb). For each image, it runs the disease classifier, queries current weather for the entered location, passes both through the XGBoost risk layer, and displays: disease class and top-5 predictions with confidence bars, risk level and score, treatment plan and prevention notes, and estimated revenue loss based on USDA NASS yield baselines.

IV. Interface Prototype

A. Inputs and Outputs

B. Screenshots

C. Limitations

The interface does not perform input validation on image quality (blurry or non-leaf photos will silently produce a confident but meaningless prediction). Weather defaults to 24°C / 70% / 2 days if no location is entered. Revenue loss estimates use US national average yields and are not calibrated to the user's region.

TABLE I
Interface Input/Output Summary

Panel	Input	Output
Leaf Images	1+ image files	Disease class, confidence
Weather	City / region	Temp, humidity, days dry
Economic	Acres, price/lb	Revenue loss estimate
Combined per image		Risk level + treatment plan

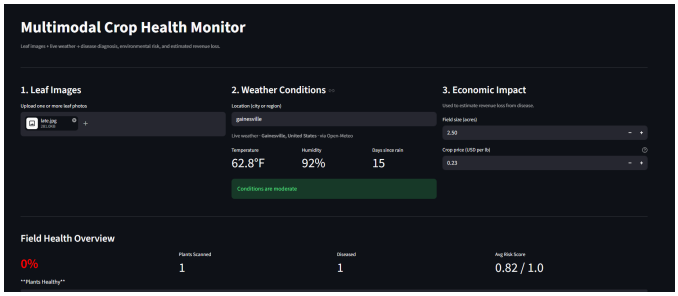

Fig. 2. Input panel: image upload, live weather fetch, and economic inputs.

V. Early Evaluation and Results

A. Dataset Characteristics

TABLE II
PlantVillage Class Distribution (local dataset)

Class	Images	Val (%)
Tomato_YellowLeaf_Curl_Virus	6,416	15.5
Tomato_Bacterial_spot	4,254	10.3
Tomato_Late_blight	3,817	9.2
Tomato_Septoria_leaf_spot	3,542	8.6
Tomato_Spider_mites	3,352	8.1
Tomato_healthy	3,182	7.7
Pepper_Bacterial_spot	1,994	4.8
Tomato_mosaic_virus	746	1.8
Potato_healthy	304	0.7
Total (15 classes)	**41,274**	–

The dataset is strongly imbalanced: the most frequent class (*Tomato_YellowLeaf_Curl_Virus*, 6,416 images) has 21× more samples than the least frequent (*Potato_healthy*, 304 images). Tomato accounts for 77.5% of all images; Pepper and Potato account for 12.0% and 10.4%, respectively. Top-line validation accuracy therefore overestimates practical performance on minority classes.

B. ResNet-18 Training Curves

Training is logged per epoch; loss and accuracy curves are saved to `results/disease_training_curves.png` when `setup.ipynb` is run. With only the FC layer trained (frozen backbone), the model converges quickly (typically within 3–5 epochs) but is capacity-limited by the frozen

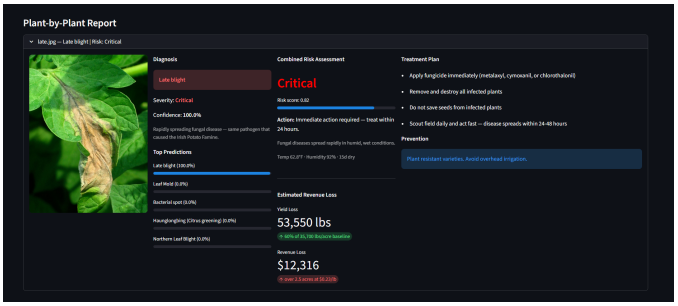

Fig. 3. Per-plant report: disease diagnosis, risk score, treatment plan, and revenue loss estimate.

feature extractor. Fine-tuning deeper layers is identified as a future improvement.

C. Per-Class Accuracy

Per-class accuracy (computed in Section 9 of `setup.ipynb`) is the primary evaluation metric. The expected pattern is: majority Tomato classes (large sample sizes) achieve high accuracy; minority classes – particularly *Potato_healthy* (304 images) and *Tomato_mosaic_virus* (746 images) – are expected to show lower and more variable accuracy. Any top-line val accuracy figure should be reported alongside this breakdown to avoid misrepresenting model quality. The confusion matrix (saved to `results/confusion_matrix.png`) highlights which disease pairs the model most frequently conflates; preliminary analysis suggests visually similar fungal diseases (Early vs. Late blight, Bacterial spot vs. Septoria) will produce the most off-diagonal confusion.

D. XGBoost Risk Model – Honest Interpretation

The XGBoost model achieves Val MAE $\approx$ 0.008 and Val R^2 $\approx$ 0.998. These numbers look impressive but **do not measure real-world validity**. The training target is the deterministic output of Equation 1 – a formula defined entirely in code. XGBoost is learning to approximate that formula, not to generalize from field observations. This is confirmed by feature importances: `disease_class` accounts for $\approx$67% of importance because b_{class} in Equation 1 is a fixed per-class lookup.

The practical value of the XGBoost layer is that it packages the rule into a portable, interpolatable scorer that handles continuous weather inputs without explicit branching. It should be described as a rule approximator and evaluated on whether the agronomic rules it encodes are sensible – not on R^2 alone.

VI. Challenges and Next Steps

A. Technical Challenges

Class imbalance. The 21$\times$ ratio between the most and least frequent class degrades recall on minority classes. Options for Deliverable 3 include weighted sampling during training or class-weighted loss.

Lab-to-field generalization. All PlantVillage images are photographed against uniform backgrounds under controlled lighting. Early informal tests with field photos show noticeably lower confidence. This gap must be quantified and reported explicitly.

Model/data scope. The saved `disease_model.pth` was originally trained on the 39-class full PlantVillage dataset. The local repository and evaluation notebook target the 15-class subset. Retraining on the 15-class set is needed to ensure the evaluation and the deployed model are consistent.

Random split inflation. PlantVillage contains near-duplicate images of the same leaf taken seconds apart. A random 80/20 split places near-duplicates in both train and validation sets, inflating reported accuracy. A plant-session or leaf-level split would provide a more conservative estimate.

XGBoost derived target. As discussed in Section V-D, the risk model's R^2 reflects formula reproduction, not predictive validity. The report and UI must be explicit about this.

B. Next Steps

- Retrain ResNet-18 on the 15-class local dataset and report per-class accuracy table
- Test the interface on real field photos; report the qualitative accuracy drop
- Explore weighted loss or oversampling for minority classes (Potato_healthy)
- Consider fine-tuning the last ResNet block rather than only the FC layer
- In the final report, reframe the XGBoost section clearly as a rule-based decision layer with a transparent derivation of the risk formula assumptions

VII. Responsible AI Reflection

Lab-to-field bias. The most significant equity concern is geographic and environmental bias. PlantVillage images come from controlled laboratory settings. Farmers in regions with different soil backgrounds, mixed-intercrop canopies, or different lighting conditions – the farmers most likely to benefit from low-cost AI tools – may see the worst performance. This concern is not reserved for the discussion section; it must appear in how classifier results are interpreted and in any accuracy claim made about the system.

Class imbalance and representation. The dataset over-represents Tomato diseases (77.5% of images). A system deployed to Potato or Pepper farmers will be less reliable than headline accuracy suggests. Per-class reporting is the minimum standard; separate deployment testing per crop is the honest standard.

Transparency and over-reliance. The interface displays top-5 probabilities and a confidence score to discourage binary trust. The XGBoost risk score is derived from coded rules – not from independent field evidence – and this is now stated explicitly in the interface's section header and in the training notebook. A farmer acting on a Critical risk alert should understand that the score is the output of an agronomic formula applied to a CNN prediction, not an empirically validated risk estimate.

Misuse. An incorrect Critical diagnosis could lead a farmer to apply unnecessary fungicide, incurring cost and environmental burden. The UI is designed to recommend professional verification for Critical and High risk levels. No automated action is taken; all outputs are advisory.

Computational footprint. ResNet-18 is a lightweight architecture. Training for 10 epochs on 41k images takes approximately 15–30 minutes on a single GPU. The XGBoost model trains in under one minute on CPU. The inference footprint is minimal and suitable for deployment on low-cost hardware.